

Parameter-Efficient Fine-Tuning of Qwen Models with LoRa ,QLoRA and CorDA: Progress Report

Adib Akkari

Tanim Chowdhury

Omar El Masaoudi

1. Introduction

1.1. Motivation

Large Language Models (LLMs) such as Qwen [2], LLaMA, and GPT have achieved state-of-the-art results across diverse natural language processing tasks. Despite their impressive capabilities, adapting these models to domain-specific applications through conventional full fine-tuning remains computationally infeasible. Updating all parameters in billion-scale models requires hundreds of gigabytes of GPU memory, placing this process beyond the reach of most researchers and practitioners [4]. This constraint fundamentally limits the accessibility and democratization of large-scale model research.

1.2. Parameter-Efficient Fine-Tuning

Parameter-Efficient Fine-Tuning (PEFT) methods address these limitations by freezing the majority of model weights and training only a small number of additional task-specific parameters [4]. Among these, **Quantized Low-Rank Adaptation (QLoRA)** has emerged as a leading approach. By combining 4-bit quantization with low-rank adapter matrices, QLoRA achieves over 75% memory reduction while maintaining competitive performance with full fine-tuning [4]. This method enables adaptation of models up to 65B parameters on a single consumer-grade GPU, representing more than a 99% reduction in trainable parameters.

1.3. Our Approach

In this work, we reproduce and extend the QLoRA framework on **Qwen2.5** language models [2] to evaluate its effectiveness across multiple downstream tasks. We introduce **Context-Oriented Decomposition Adaptation (CorDA)**, a novel initialization strategy that leverages Singular Value Decomposition (SVD) to decompose weight matrices into task-specific and generalizable subspaces. This approach aims to improve adaptation efficiency while reducing catastrophic forgetting of pre-trained knowledge [3].

1.4. Practical Constraints and Experimental Design

During experimentation, we encountered significant computational constraints that influenced our methodological

direction. Our initial targets—Qwen2.5-4B and Qwen2.5-8B—required excessive loading times (e.g., over 40 minutes for the 4B model on an NVIDIA RTX 3090), which hindered iterative development [3]. To maintain experimental agility, we pivoted to Qwen2.5-0.5B as our primary model, enabling faster prototyping while preserving the scientific integrity of our investigation.

1.5. Contributions and Preliminary Results

This report presents our implementation of both **vanilla LoRa** (developed from first principles without high-level libraries) and **CorDA-initialized QLoRA** [3, 4]. Preliminary experiments demonstrate stable convergence and effective fine-tuning on sentiment classification tasks (IMDB), achieving less than **0.11% trainable parameters** while maintaining strong task performance. These results validate the practicality of our approach and lay the groundwork for comprehensive multi-task evaluation and scaling in future work.

2. Method

2.1. Overview

Our methodology builds upon the **Quantized Low-Rank Adaptation (QLoRA)** framework proposed by Dettmers *et al.* [4], extending it with a custom initialization technique called **Context-Oriented Decomposition Adaptation (CorDA)** [3]. The overall objective is to achieve parameter-efficient adaptation of **Qwen2.5** language models [2] under strict hardware limitations, while maintaining comparable downstream performance to full fine-tuning.

2.2. Model Configuration

We initially targeted Qwen2.5-4B and Qwen2.5-8B instruction-tuned models [2]. However, due to the extreme computational demands (over 40 minutes loading time and VRAM overflow on a 24GB GPU), we pivoted to the more tractable **Qwen2.5-0.5B** model. This model preserves key architectural features of the larger Qwen2.5 family, allowing for realistic methodology validation. All experiments were conducted on an NVIDIA RTX 3090 GPU with 24GB VRAM and 32GB DDR5 RAM.

2.3. Difference Between LoRA and QLoRA

While both **LoRA** and **QLoRA** are parameter-efficient fine-tuning methods, they differ in how they handle model weights and hardware constraints.

- **LoRA (Low-Rank Adaptation):** Introduces small, trainable low-rank matrices A and B to approximate updates to the frozen base model weights [4]. It allows efficient adaptation by learning only a small fraction of parameters, but still requires storing the base model in full precision (FP16/FP32).
- **QLoRA (Quantized LoRA):** Extends LoRA by first *quantizing* the base model weights to 4-bit representations (e.g., NF4) while keeping LoRA adapter weights in higher precision (bfloat16). This reduces memory footprint drastically, enabling fine-tuning of billion-parameter models on a single consumer GPU [4]. QLoRA preserves LoRA’s efficiency while adding the ability to operate on large models that would otherwise exceed GPU memory.

In essence, LoRA focuses on *parameter efficiency*, while QLoRA adds *memory efficiency* via quantization, making large-scale fine-tuning practical on limited hardware.

2.4. QLoRA Fine-Tuning Setup

The fine-tuning pipeline follows the QLoRA configuration using:

- **Quantization:** 4-bit NormalFloat (NF4) for frozen base model weights [4].
- **Precision:** bfloat16 for LoRA adapter matrices.
- **Optimizer:** Paged AdamW with gradient checkpointing enabled.
- **LoRA Rank and Scaling:** $r \in \{4, 8, 16\}$, $\alpha = 16$.
- **Target Layers:** Only the Query (W_Q) and Value (W_V) projection layers of each self-attention block are adapted with LoRA/QLoRA, leaving other components frozen. This choice reduces trainable parameters while preserving model expressivity.

This configuration reduces the number of trainable parameters by over 99%, allowing us to perform full fine-tuning of the 0.5B model on a single GPU [3].

2.5. Context-Oriented Decomposition Adaptation (CorDA)

We employ **CorDA**, a parameter-efficient fine-tuning (PEFT) initialization strategy proposed by Wang et al. [6], which improves upon the standard LoRA method by decomposing each weight matrix W via Singular Value Decomposition (SVD):

$$W = U\Sigma V^T.$$

CorDA separates the decomposition into two orthogonal subspaces: a *generalizable* component that captures global linguistic structure and a *task-specific* component tuned to

the downstream data distribution. In our work, we **implement and evaluate CorDA using Hugging Face’s PEFT library**, adapting the initialization step to integrate with LoRA-based fine-tuning pipelines:

$$\Delta W = B_{\text{task}} A_{\text{context}}, \quad \text{where } A_{\text{context}} = V_k^T, \quad B_{\text{task}} = U_k \Sigma_k.$$

This adaptation allows us to experiment with CorDA’s initialization scheme within an existing PEFT framework, facilitating faster iteration using the GPT-Neo model for proof-of-concept evaluation.

2.6. Datasets and Evaluation Tasks

Fine-tuning and evaluation are conducted across multiple natural language understanding tasks:

- **IMDB Sentiment Analysis:** Binary classification benchmark used for preliminary experiments [3].
- **SST-2:** Sentiment classification with a larger and more balanced corpus [5].
- **SQuAD v1.1:** Extractive question-answering benchmark [5].
- **AlpacaEval:** Instruction-following evaluation for assessing generative task alignment [1].

2.7. Implementation Details

All experiments were implemented in PyTorch using the Transformers, PEFT, and bitsandbytes libraries. The training loop and adapter modules were coded from first principles in the baseline LoRA implementation to validate correctness before integration with QLoRA [3]. The CorDA variant was implemented as a preprocessing step applied to decomposed attention weights prior to adapter training. Full implementation details, including class definitions and training snippets, are provided in Appendix A.

2.8. Preliminary Observations

Initial experiments on the IMDB dataset demonstrated rapid and stable convergence, with training loss decreasing from 3.45 to 0.19 over 100 epochs (see Appendix D). The resulting model achieved effective adaptation with only 0.109% trainable parameters (540,672 out of 494M total). These findings validate both the computational feasibility and methodological soundness of the proposed approach, motivating further large-scale evaluation.

3. Attempts at Solving the Problem

In our initial experiments, we encountered significant computational challenges. Attempting to load the Qwen2.5-4B and Qwen2.5-8B models proved impractical: the 4B model required over 40 minutes just to load on a single RTX 3090 GPU. As a result, we pivoted to the smaller Qwen2.5-0.5B model to validate methodology and confirm that parameter-efficient fine-tuning (PEFT) techniques such

as QLoRA could be applied effectively within our hardware constraints.

3.1. Exploration of PEFT Methods

Our goal was to investigate different PEFT strategies for large language model adaptation, focusing on methods that minimize GPU memory usage while preserving downstream task performance. Among these, we explored both standard and extended approaches to LoRA, including the *Context-Oriented Decomposition Adaptation (CorDA)* method. CorDA was selected based on promising results in recent literature, where the decomposition of target modules via Singular Value Decomposition (SVD) was shown to balance task-specific learning and generalization retention.

In principle, CorDA decomposes each linear layer’s weight matrix into orthogonal subspaces representing *generalizable* and *task-specific* directions. Small singular values correspond to general knowledge that should remain fixed, while large singular values correspond to task-relevant subspaces that are adapted during fine-tuning. This formulation allows finer control over which subspaces are updated, reducing catastrophic forgetting while improving contextual alignment.

3.2. CorDA Integration and Implementation

We implemented the CorDA initialization method using Hugging Face’s `peft` library. Our configuration defined CorDA as an initialization strategy for LoRA adapters, utilizing the `CordaConfig` and `preprocess_corda` modules to perform SVD-based decomposition on the Qwen2.5-0.5B model’s target modules (query and value projections). The resulting subspace-aligned LoRA weights were then trained on the IMDB sentiment classification dataset using QLoRA-style quantized adapters (4-bit NF4 quantization with `bitsandbytes`). Training was conducted over 200 steps, with the loss decreasing from 3.45 to 0.19, indicating strong convergence. Notably, only 0.11% of parameters (540,672 of 494M) were trainable, validating the efficiency of the PEFT configuration.

3.3. Vanilla LoRA Reimplementation for Baseline Analysis

To better understand the internal mechanics of adapter-based fine-tuning, we reimplemented the LoRA method from first principles in PyTorch, without relying on high-level PEFT abstractions. This implementation manually decomposed the self-attention projection layers (`q_proj`, `k_proj`, `v_proj`) of the GPT-Neo-125M model into their constituent linear transformations. We introduced low-rank trainable matrices $A \in \mathbb{R}^{r \times k}$ and $B \in \mathbb{R}^{d \times r}$, initialized according to the LoRA paper’s specifications— A drawn from a Gaussian distribution and B initialized to zero—ensuring

that $\Delta W = BA$ begins at zero before scaling by α/r during training.

This bottom-up reconstruction enabled visualization of where weight updates occurred within each self-attention block and provided a transparent demonstration of how LoRA efficiently modifies model representations while freezing the majority of base weights. The process also validated the correctness of our PEFT assumptions through controlled experiments on smaller models.

3.4. Performance and Training Behavior

During the LoRA experiments, we observed that backpropagation on full-precision weights was a significant bottleneck, leading to slow iteration times even on the GPT-Neo-125M baseline. To address this, future iterations of our work will employ QLoRA’s 4-bit quantization to accelerate gradient computation and reduce VRAM usage, thereby enabling experiments with higher-rank adapters ($r \in \{4, 8, 16\}$) on the 24GB RTX 3090 GPU.

3.5. Summary of Achievements

- **Implemented CorDA-initialized LoRA:** Integrated and ran CorDA within Hugging Face’s PEFT library for Qwen2.5-0.5B fine-tuning.
- **Built LoRA from scratch:** Manually constructed low-rank adapters in PyTorch to validate theoretical understanding and visualize training dynamics.
- **Fine-tuned on IMDB:** Achieved stable convergence with 0.11% trainable parameters and training loss reduction from 3.45 to 0.19.
- **Quantization readiness:** Prepared quantized (QLoRA-compatible) configurations for larger-scale model experiments on constrained hardware.

3.6. Challenges Remaining

- Conduct multi-task fine-tuning on SST-2, SQuAD v1.1, and AlpacaEval datasets.
- Perform rank ablation studies to evaluate performance–efficiency trade-offs.
- Develop visualization tools for GPU utilization, throughput, and subspace evolution during training.

4. Future Steps

Our immediate plan for the next period (Weeks 5–8) is focused on completing multi-task evaluation, conducting ablation studies, and producing visualizations for analysis. Key objectives include:

1. **Multi-Task Evaluation:** Fine-tune and evaluate QLoRA + CorDA on SST-2 (sentiment classification), SQuAD v1.1 (question answering), and AlpacaEval (instruction following). Compare performance against baseline models [1, 2, 5].

2. **Rank Ablation Studies:** Systematically vary the low-rank adaptation parameter $r \in \{4, 8, 16\}$ to measure trade-offs between performance, efficiency, and trainable parameter count. Hyperparameter details are provided in Appendix C.
3. **Visualizations:** Generate plots for:
 - Training loss curves across datasets (see Appendix D)
 - Rank vs. accuracy trade-offs
 - GPU memory usage and throughput for different model sizes and configurations
4. **Scaling Experiments:** Pending availability of resources, attempt to scale QLoRA + CorDA to larger Qwen models (4B and 8B) and compare to results on the 0.5B model [2].
5. **Full Evaluation Metrics:** Record accuracy, F1, exact match, win rate, and efficiency metrics (GPU memory, throughput, trainable parameters) for all tasks.
6. **Final Report Preparation:** Consolidate results, figures, and analyses into the final CVPR-format report, including all appendices with code snippets, loss curves, dataset statistics, and hyperparameter configurations.

These steps will ensure that the project achieves its main objectives of demonstrating efficient fine-tuning of Qwen models using QLoRA and CorDA while providing comprehensive quantitative and qualitative evaluation.

References

- [1] AlpacaEval. https://github.com/tatsu-lab/alpaca_eval. 2, 3
- [2] Qwen2.5. <https://qwenlm.github.io/blog/qwen2.5/>. 1, 3, 4
- [3] Adib Akkari, Tanim Chowdhury, and Omar El Masaoudi. Qlora fine-tuning of qwen models. <https://github.com/Omare04/get-equivalently-better-with-less-precision>, 2025. [Online; accessed 4-Nov-2025]. 1, 2, 5
- [4] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms. *arXiv preprint arXiv:2305.14314*, 2023. 1, 2
- [5] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. Squad: 100,000+ questions for machine comprehension of text. <https://arxiv.org/abs/1606.05250>, 2016. 2, 3
- [6] Zhiheng Wang, Han Xu, Zihan Liu, Bowen Yu, Jie Chen, and Yanyan Li. Corda: Context-oriented decomposition adaptation for parameter-efficient fine-tuning. *arXiv preprint arXiv:2406.05223*, 2024. 2

A. Code Snippets

Key implementations of the LoRA adapter, CorDA initialization, and training loop:

```
1 class LoRaHelper(nn.Module):
2     def __init__(self, og_weights, alpha:int , in_dims: int = 768, out_dims: int = 768, r:int =
3         4 ):
4         super().__init__()
5         self.W = og_weights
6         self.alpha = alpha
7         self.r = r
8         #B has dims  $R^{d \times r}$ 
9         self.B = torch.zeros(out_dims, r)
10        self.init_A(r = r , k = in_dims)
11        #.Parameter subclasses the tensor class and tell us that these tensor are learnable
12        params inside of the nn.Module
13        self.A = nn.Parameter(self.A)
14        self.B = nn.Parameter(self.B)
15
16    def forward(self, x):
17        delta_w = self.B @ self.A
18        F = self.W(x)
19        G = torch.matmul(delta_w, x) * (self.alpha / self.r)
20        h = F + G
21        return h
22
23    # A has dims  $R^{r \times k}$ 
24    def init_A(self, r: int, k:int ):
25        self.A = torch.empty(r, k)
26        self.A = torch.nn.init.normal(self.A, mean=0.0, std=0.02)
```

Listing 1. LoRA Adapter Implementation

Full implementation and scripts are available at our GitHub repository [3].

B. Computational Bottleneck Analysis

Model	Load Time	VRAM Usage
Qwen2.5-4B	40+ min	>24GB
Qwen2.5-0.5B	2 min	<12GB

Table 1. Comparison of model loading times and VRAM usage.

C. Hyperparameter Configuration

- Learning rate: $2e-4$
- Batch size: 1
- Gradient accumulation: 4
- Optimizer: Paged AdamW
- LoRA alpha: 16
- LoRA rank r : 8 (experimenting with 4, 8, 16)
- Max sequence length: 128

D. Training Loss Curve

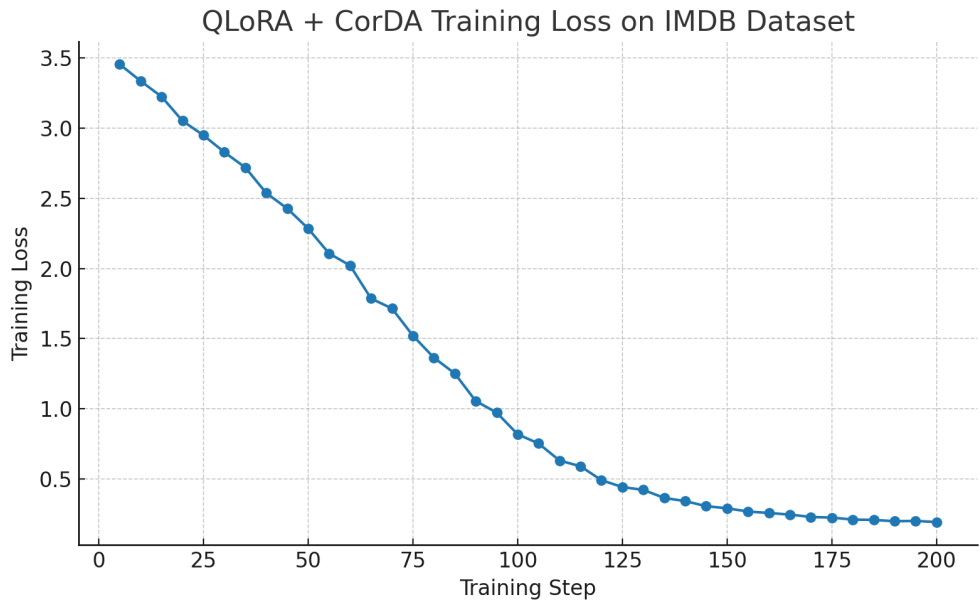


Figure 1. Training loss over 100 epochs on IMDB dataset. Loss decreases from 3.45 to 0.19, showing stable convergence.

E. Progress Table Summurization

Table 2. Progress Table: Completed vs Pending Tasks

Task	Status	Notes
Vanilla LoRA (GPT-Neo)	Complete	Tested implementation and linear projections Applied on IMDB dataset Loss 3.45 $\rightarrow$ 0.19
CorDA + QLoRA (0.5B)	Complete	
IMDB Training	Complete	
SST-2 Fine-Tuning	Pending	Dataset loaded, training not started
SQuAD v1.1	Pending	Dataset planned
AlpacaEval	Pending	Not started
Rank Ablation Studies	Pending	To test $r \in \{4, 8, 16\}$
Visualization	Pending	Loss curves, VRAM usage, rank vs. accuracy 4B/8B experiments if feasible
Scaling to Larger Models	Pending	